Convolutional Neural Networks: Introduction

2026-04-17

Introduction

Convolutional Neural Networks (CNNs; LeCun, 1989) replace fully-connected layers with local, weight-sharing convolutional filters. The shared weights encode translation invariance: a feature detector (edge, texture) learned once applies at every spatial location. CNNs dominate image classification, segmentation, and many signal-processing tasks.

Prerequisites

Feedforward networks; convolutions and cross-correlations; image tensors.

Theory

Convolutional layer: a set of learned filters K_c; each filter slides across the input, producing a feature map. Output channel count equals number of filters.

Pooling: max or average over a spatial window; reduces dimensionality and increases receptive field.

Typical architecture (classic): conv -> ReLU -> pool, repeated; then flatten to fully-connected for classification. Modern architectures (ResNet, EfficientNet) add residual connections and batch normalisation.

Assumptions

Inputs have spatial structure (images, spectrograms, time series); sufficient training data or a pretrained backbone.

R Implementation

```
library(torch)

torch_manual_seed(2026)

# Simulate a small batch of "images"
x <- torch_randn(4, 1, 28, 28)          # 4 images, 1 channel, 28x28
y <- torch_randint(0, 10, size = list(4), dtype = torch_long())

# Simple CNN: 2 conv layers + FC head
cnn <- nn_module(
  initialize = function() {
    self$conv1 <- nn_conv2d(1, 16, kernel_size = 3, padding = 1)
    self$conv2 <- nn_conv2d(16, 32, kernel_size = 3, padding = 1)
    self$pool  <- nn_max_pool2d(2)
    self$fc1   <- nn_linear(32 * 7 * 7, 64)
    self$fc2   <- nn_linear(64, 10)
  },
  forward = function(x) {
```

```
    x <- self$pool(nnf_relu(self$conv1(x)))
    x <- self$pool(nnf_relu(self$conv2(x)))
    x <- x$view(c(x$size(1), -1))
    nnf_relu(self$fc1(x)) %>% self$fc2()
  }
)
model <- cnn()
logits <- model(x)
logits$shape                # [4, 10]
```

Output & Results

Logits of shape [batch, classes] ready for a cross-entropy loss during training.

Interpretation

"A two-layer CNN produced 10-class logits for a batch of 28x28 inputs. In a real MNIST pipeline this architecture would reach ~99 % accuracy after training."

Practical Tips

- Pre-trained CNN backbones (ResNet50, EfficientNet) with fine-tuning outperform training from scratch for most tasks with $<$ 1M images.
- Data augmentation (flip, rotate, crop) is essential for image classification; use `torchvision::transform_*`.
- Batch normalisation accelerates training and stabilises gradients; include after every conv layer.
- For 1-D signals (ECG, audio), use 1-D convolutions (`nn_conv1d`).
- Vision transformers (ViT) now rival or exceed CNNs on large datasets; CNNs remain preferred for small-to-moderate data.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis